

# Operating System: A Few Recap

Dr. Jit Mukherjee

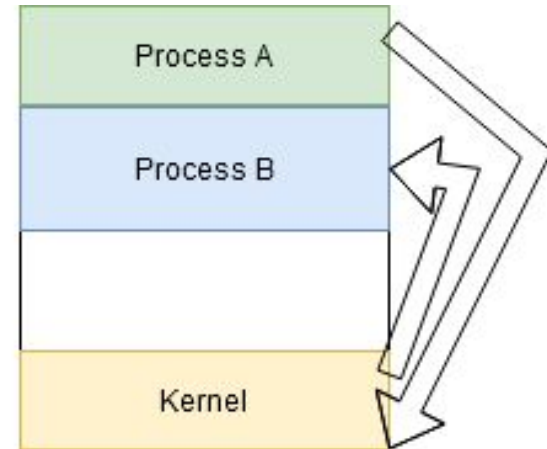
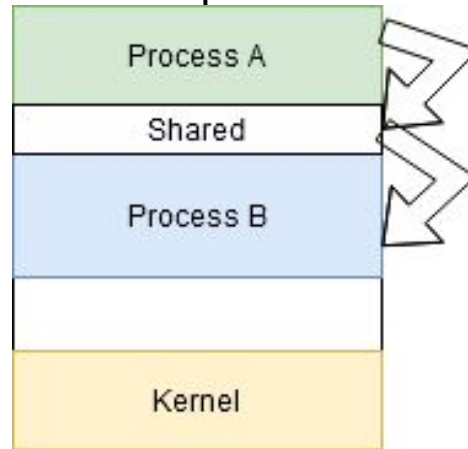


# System Call

- A system call is a mechanism that provides the interface between a process and the operating system
- System calls are the only entry points for the kernel system. The kernel system can only be accessed using system calls.
- A system call is a method for a computer program to request a service from the kernel of the operating system
- The Application Program Interface (API) connects the operating system's functions to user programs. It acts as a link between the operating system and a process, allowing user-level programs to request operating system services.
- Type
  - Process Control
    - creating, load, abort, end, execute, process, terminate the process
  - File Management
    - creating files, delete files, open, close, read, write, etc.
  - Device Management
    - read, device, write, get device attributes, release device, etc.
  - Information Maintenance
    - getting system data, set time or date, get time or date, set system data, etc.
  - Communications
    - create, delete communication connections, send, receive messages, etc.

# Interprocess Communication

- A process is independent if it can not affect or be affected by others
- A process is cooperating if can affect or be affected by others.
- Any process that share data with other processes
- Why we need
  - Information sharing
  - Computation speedup
  - Modularity
  - Convenience
- To exchange info or data
  - Shared Memory
  - Message Passing
- Many system implements both
  - Message Passing - No conflict, easy to implement, good for small data
  - Shared - maximum speed



# Shared Memory

- Shared memory resides in the address space of the parent process
- Any process needs this for communication, add the address space
- Systems calls are required to shared memory regions
- Producer Consumer Problem - A producer process provides info that is consumed by consumer
  - Producer and Consumer process runs concurrently using a buffer filled by producer and emptied by consumer
  - Synchronized
  - Unbounded buffer - no practical limit to size
  - Bounded buffer - fixed buffer size
- Circular array, Circular LL

# Message Passing

- Communication and sync without sharing same address space
  - Distributed environment
- Message passing systems are implemented using system calls, needs kernel intervention
- At least two operation - Send and Receive
- Message can be of variable or fixed size.
  - Fixed - straightforward implementation but restricted
  - Variable - complex implementation
- Communication Link (Logical implementation)
  - Direct or Indirect Communication
    - Name the process or have a port or mailbox
  - Synchronous or asynchronous communication
    - Sender and Receiver names each other/ Sender names the Receiver
  - Automatic or explicit buffering
    - Indefinite length or explicitly mentioned size

# Synchronization and Buffering

- Message passing be either blocking or non blocking
  - Blocking Send - the sending process is blocked until the message is received
  - Nonblocking Send - Sender process send the message and resumes operations
  - Blocking Receive - The receiver blocks until a message is available
  - Nonblocking Receive - The receiver retrieves either a valid message or a null
- Producer Consumer problem becomes trivial with blocking send and receive
- Messages resides in a temporary queue.
  - Zero Capacity - the queue has a max length zero.
    - No message waiting. Sender must block until receiver receives
    - Message passing with no buffering
  - Bounded Capacity - Finite length. At most n number messages can reside
  - Unbounded Capacity - Queue length is potentially infinite

# Communication in Client Server System

- Sockets
  - Socket is an endpoint for communication. It has an IP address with a port number
  - A pair of sockets - one for each process
  - Servers waits for incoming client request to a specific port
  - Once request received, server accepts the connection. Telnet - Port 23, ftp - Port 21, http - port 80
- Remote Procedure Calls
  - A client has a request message (a procedure or a function call) that the RPC translates and sends to the server.
  - When the server receives the request, it sends the required response back to the client.
  - The client is blocked while the server is processing the call and only resumed execution after the server is finished.
- Remote Method Invocation
  - Java feature similar to RPC. Object oriented, can pass object
  - Remote object in client (Stub) and server (Skeleton)